Plan de mejoramiento: backend con IA y consumo en React

Práctica en parejas para construir, durante aproximadamente una semana, una app con backend en Django, frontend en React y una funcionalidad que integra un servicio externo de IA.

Datos generales y objetivo

El trabajo se hace en parejas, dura aproximadamente una semana y se desarrolla con Git en un repositorio compartido.

- **Modalidad:** parejas. Cada pareja construye una app propia sobre un tema distinto al de las demás.
- **Git:** un repositorio compartido donde ambos integrantes tienen commits propios, con mensajes que expliquen qué se hizo.
- **Tecnologías:** Django (API REST) para el backend y React con Vite y TypeScript para el frontend.

Objetivo: construir una app corta con backend en Django (API propia), frontend en React y una funcionalidad que integra un servicio externo de IA a través de la API propia, y presentarla en la sustentación.

Temas

Cada pareja elige uno de estos 5 temas de actualidad, sin repetirse entre parejas. Cada tema propone 4 recursos, que son los 4 CRUDs mínimos del backend, y una función de IA.

Tema	Recursos (CRUDs)	Qué hace la IA
Gestión de emergencias por desastres naturales	Zonas, Eventos, Reportes ciudadanos, Albergues	Clasifica el reporte (tipo y urgencia) y lo resume
Prevención y plan de emergencia	Tipos de riesgo, Protocolos, Elementos del kit, Planes familiares	Genera un plan de emergencia personalizado en formato estructurado
Orientación en seguridad ciudadana	Municipios, Instituciones y líneas de atención, Comunicados verificados, Preguntas frecuentes	Resume comunicados y responde preguntas sobre dónde acudir

Tema	Recursos (CRUDs)	Qué hace la IA
Ambiente y clima	Zonas, Mediciones, Alertas, Recomendaciones	Interpreta las mediciones y sugiere recomendaciones a la comunidad
Ayuda humanitaria y voluntariado	Campañas, Voluntarios, Insumos, Puntos de acopio	Prioriza necesidades y resume el estado de una campaña

Los recursos son una guía: cada pareja puede ajustar los campos y las relaciones, siempre que cumpla los 4 CRUDs. En el tema de seguridad ciudadana no se manejan datos personales de víctimas ni de terceros, solo información verificada y de orientación.

Qué se debe construir

La app tiene tres partes: una API propia en Django, una funcionalidad de IA dentro de esa API y un frontend en React que consume solo la API propia.

Backend (Django)

- Mínimo **4 CRUDs** con modelos relacionados entre sí.
- Validaciones de los datos, códigos HTTP correctos y manejo de errores.
- Documentación de todos los endpoints con **Swagger**.

Funcionalidad de IA (proxy)

El frontend nunca llama a la IA directamente. El flujo es este:

1. El frontend envía datos a un endpoint de la API propia.
2. El backend valida la entrada y arma el prompt en el servidor.
3. El backend llama a la API externa de IA (OpenAI, DeepSeek, Gemini, MiMo u otra) con la cuenta de uno de los integrantes.
4. El backend procesa la respuesta: la valida, la transforma y la guarda (por ejemplo, pidiéndole a la IA un formato estructurado como JSON).
5. El backend devuelve el resultado en el formato de su propia API.

Cada aprendiz usa su propia cuenta; quien no tenga, puede usar la capa gratuita de cualquier proveedor.

Frontend (React + Vite + TypeScript)

- Puede partir de la plantilla MatDash, de otra plantilla a elección o de una hecha desde cero.
- Consume únicamente la API propia.
- Es responsivo y cuida la UI y la UX: estados de carga, mensajes de error claros y navegación sencilla.

Uso de la IA y bitácora de prompts

El código del backend debe escribirse sin ayuda de IA; en el resto del proyecto se puede usar, siempre que quede registrado en la bitácora.

- **Backend:** no se permite usar IA para generar ni corregir su código. La IA externa que integran como funcionalidad (el proxy) sí hace parte del proyecto, pero no sirve para escribirlo.
- **Frontend, documentación, presentación y demás:** se permite usar IA, pero cada prompt debe quedar registrado.
- **Bitácora individual:** cada aprendiz lleva la suya, de inicio a fin del proyecto, como un archivo dentro del repositorio.

Cada entrada de la bitácora incluye:

1. Fecha y herramienta o modelo usado.
2. El prompt que escribió.
3. Un resumen de lo que respondió la IA.
4. Qué hizo con esa respuesta: la usó, la modificó o la descartó, y por qué.

Si usaron skills o agentes de IA, deben poder explicar en la sustentación cuáles fueron y cómo los usaron.

Seguridad: variables de entorno (.env)

La clave de la API de IA nunca debe quedar escrita en el código ni subirse al repositorio. Cada pareja debe investigar cómo usar un archivo `.env` en un proyecto Django e implementarlo, y debe poder explicar en la sustentación cómo lo hizo y por qué.

Como mínimo, el proyecto debe cumplir esto:

- Las claves y datos sensibles se leen desde variables de entorno.
- El archivo `.env` no se sube al repositorio.
- El repositorio incluye un archivo de ejemplo (por ejemplo `.env.example`) sin valores reales.

- El frontend nunca recibe ni usa la clave de la IA.

Entrega y sustentación

Todo se entrega en el repositorio y se califica en la sustentación, que se hace en el ambiente de formación.

Qué debe tener el repositorio

- El código del backend y del frontend.
- La documentación de la API con Swagger.
- Las bitácoras de prompts, una por aprendiz.
- La presentación del proyecto.
- El archivo `.env.example` y las instrucciones para ejecutar el proyecto.

Presentación

Una presentación sencilla con lo esencial:

1. Objetivo del proyecto.
2. Actores.
3. Impactos.
4. Entregables.
5. Skills de IA utilizados, si los usaron, y cómo fueron utilizados.

Durante la sustentación

La pareja muestra la app funcionando y cada integrante responde preguntas técnicas sobre el backend, el uso de Git, la integración con la IA y el manejo del `.env`. Todo lo que se evalúa se verifica en este momento.

Evaluación

La nota se calcula sobre 5,0 y la sustentación vale el 50%. Cada criterio se califica en tres niveles: **completo** (100% de los puntos), **parcial** (aprox. 50%) y **no cumple** (0%).

Componente	Criterio	Completo (100%)	Parcial (50%)	No cumple (0%)	Puntos (sobre 5,0)
Sustentación (50%)	Presentación del proyecto	Presentan objetivo, actores, impactos y entregables con claridad	Falta alguno de los elementos o la presentación es confusa	No presentan el proyecto	0,50
	Demostración de la app	La app funciona de principio a fin durante la demostración	Funciona con fallas o solo en parte	No funciona o no la pueden mostrar	0,50
	Respuestas técnicas de cada integrante	Ambos explican con claridad su parte del backend, Git, la integración con la IA y el `.env`	Responden de forma incompleta o con dudas	No pueden explicar el código que entregaron	1,00
	Skills de IA y bitácora	Explican qué skills usaron y cómo, o aclaran que no usaron	Explican de forma parcial	No pueden explicar el uso de la IA que registraron	0,50
Backend (20%)	CRUDs y relaciones	Mínimo 4 CRUDs funcionales y relacionados entre sí	2 o 3 CRUDs, o sin relaciones entre ellos	Menos de 2 CRUDs funcionales	0,35
	Validaciones, errores y códigos HTTP	Valida los datos, maneja errores y responde con los códigos correctos	Cumple solo en algunos endpoints	Sin validaciones ni manejo de errores	0,25
	Documentación con Swagger	Todos los endpoints documentados y probables desde Swagger	Documentación incompleta	Sin Swagger	0,20
	Variables de entorno (.env)	Claves en `.env`, fuera del repositorio y con `.env.example`	Usa `.env` pero falta alguna de las buenas prácticas	Clave escrita en el código o subida al repositorio	0,20
IA como proxy (10%)	Flujo del proxy	El frontend llama a la API propia y esta llama a la IA con el prompt armado en el servidor	El flujo funciona pero parte de la lógica está en el frontend	El frontend llama directamente a la IA o no hay integración	0,25

Componente	Criterio	Completo (100%)	Parcial (50%)	No cumple (0%)	Puntos (sobre 5,0)
	Procesamiento de la respuesta	Valida, transforma, guarda y devuelve la respuesta en el formato de su API	Solo reenvía la respuesta con cambios mínimos	Reenvía la respuesta de la IA sin procesarla	0,25
Frontend (10%)	Consumo de la API propia	Consume solo la API propia con React y TypeScript	Consume la API con errores o sin tipado	No consume la API	0,25
	Responsividad y UI/UX	Se adapta a distintos tamaños, con estados de carga y errores claros	Responsivo solo en parte o con UX básica	No es responsivo	0,25
Git y bitácora (10%)	Uso de Git	Ambos integrantes tienen commits propios con mensajes claros	Uno aporta muy poco o los mensajes son genéricos	Solo un integrante tiene commits	0,25
	Bitácoras individuales	Una por aprendiz, completa y de inicio a fin	Incompleta o con entradas faltantes	Sin bitácora	0,25
Total					**5,00**